

Data Structure

Trees

Binary Trees

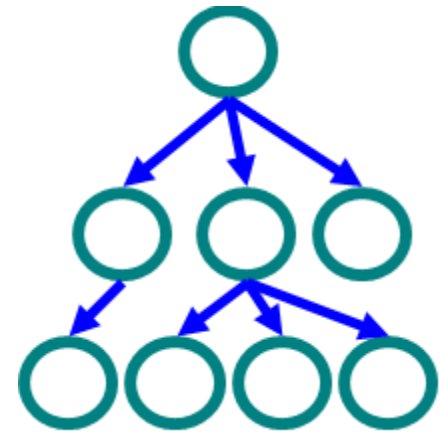
Abdul Mateen
Assistant Professor
Department of Computer Science, PU

Most of the content taken from the slides of Anna Fariha
Department of CSE, University of Dhaka

Tree as a Data Structure

Tree is a nonlinear data structure

- Contain a distinguished node R , called the root of tree and a set of sub trees.
- Two nodes n_1 and n_2 are called siblings if they have the same parent node.
- Tree:
 - has nodes
 - each node can have 0 or more **children**
 - a node can have at most one **parent**



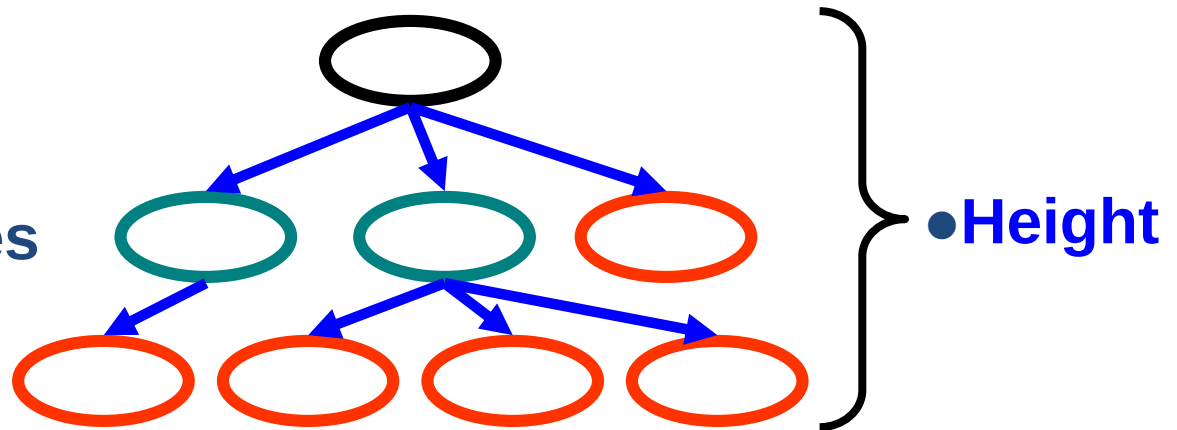
Trees

- Terminology
 - Root $\Rightarrow$ no parent
 - Leaf $\Rightarrow$ no child
 - Interior $\Rightarrow$ non-leaf
 - Depth $\Rightarrow$ distance from root to leaf

• Root node

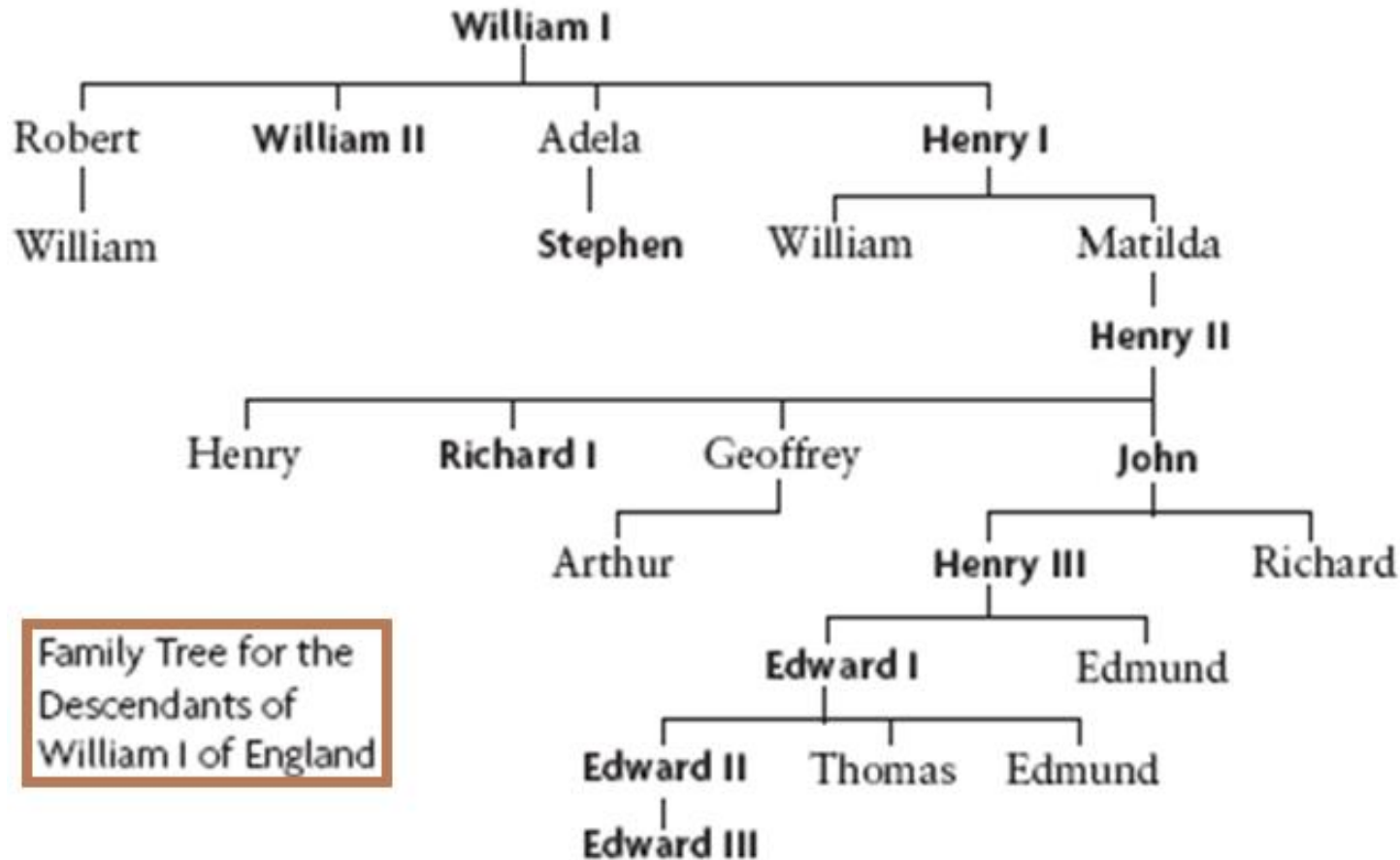
• Interior nodes

• Leaf nodes

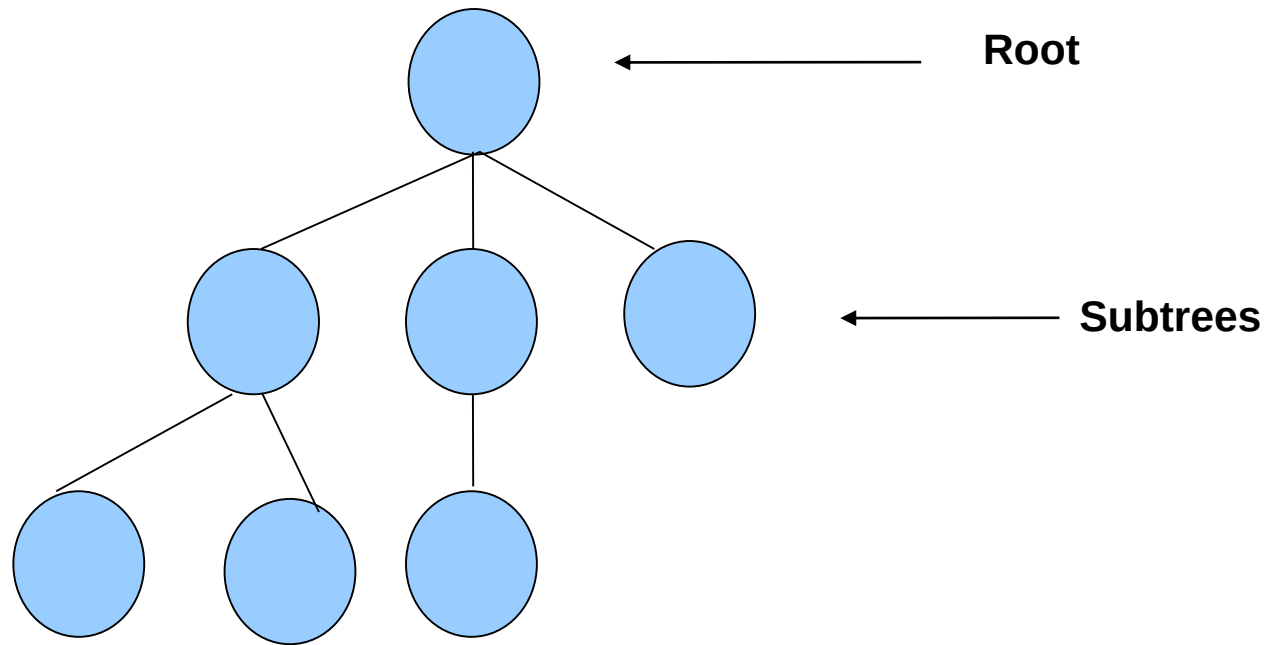


General (Non-Binary) Trees

- Nodes can have any number of children



Tree



Most Important Terminology

- Node A is the **parent** of node B if node B is a **child** of A
- Node A is an **ancestor** of node B if A is a parent of B, or if some child of A is an ancestor of B
 - In less formal terms, A is an ancestor of B if B is a child of A, or a child of a child of A, or a child of a child of a child of A, etc.
- Node B is a **descendant** of A if A is an ancestor of B
- Nodes A and B are **siblings** if they have the same parent

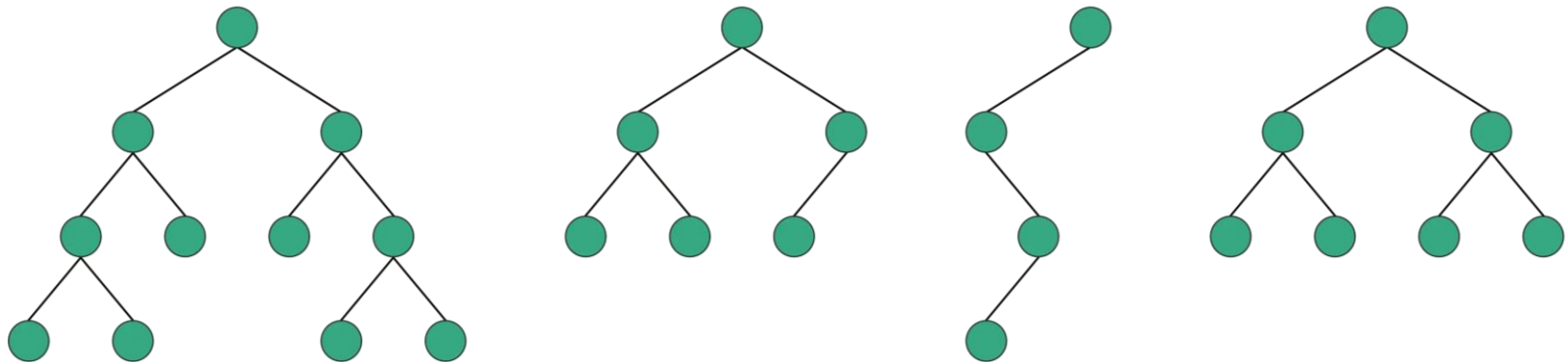
Most Important Terminology

- A tree is a collection of elements (**nodes**)
- Each node may have 0 or more **successors**
 - (Unlike a list, which has 0 or 1 successor)
- **Each node has exactly one predecessor**
 - Except the starting / top node, called the root
- Links from node to its successors are called **branches**
- Successors of a node are called its children
- Predecessor of a node is called its parent
- Nodes with no children are called **leaves**

Binary Tree

A binary tree is a tree, where nodes have only zero, one or two child nodes.

There are four examples of different binary trees. In each of them, nodes have either zero, one or two nodes only.

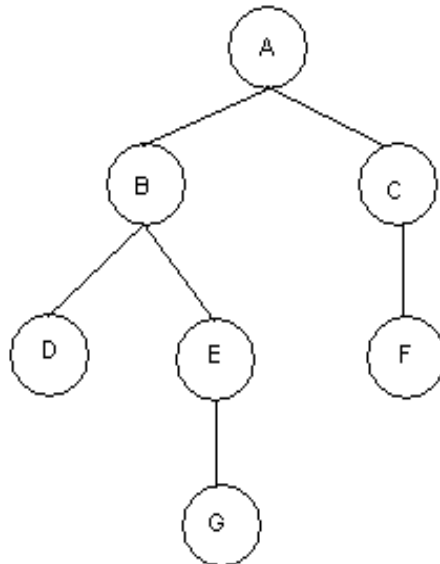


Parts of a binary tree

- Root
- Left Child
- Right Child
- Left Subtree
- Right Subtree
- **Parent**
- **Child**
- **Sibling**
- Edge
- Path
- Branch

- Height/Depth
- Level
- **Descendant**
- **Ancestor**

- Any node N in a binary tree T has either 0, 1 or 2 successors.
- Nodes with no successors are called terminal nodes or leaf nodes.



Binary Tree

Root: A

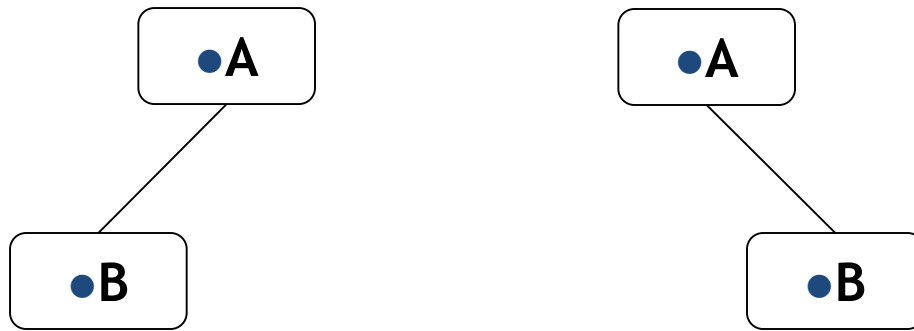
Nodes with 2 Successors: A, B

Nodes with 1 Successors: C, E

Terminal Nodes: D, F, G

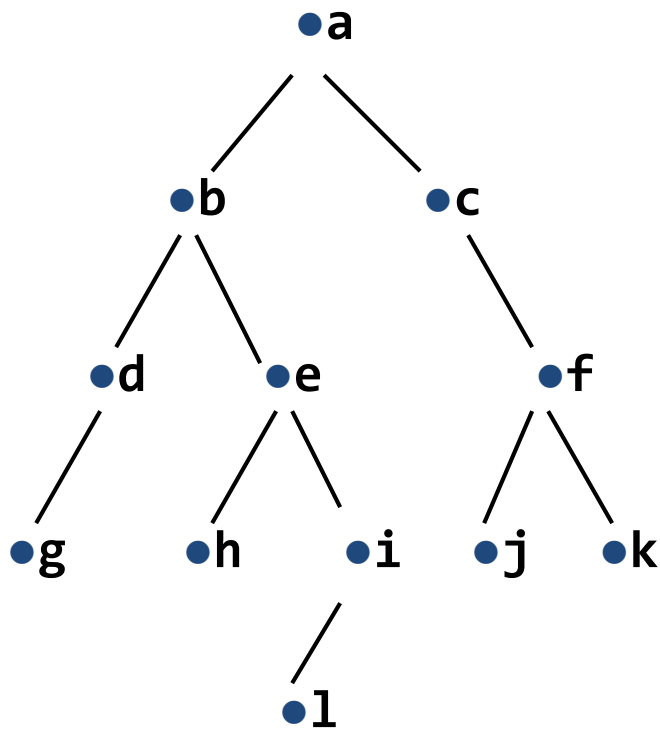
Left $\neq$ Right

- The following two binary trees are *different*:



- In the first binary tree, node A has a left child but no right child; in the second, node A has a right child but no left child
- Put another way: Left and right are *not* relative terms

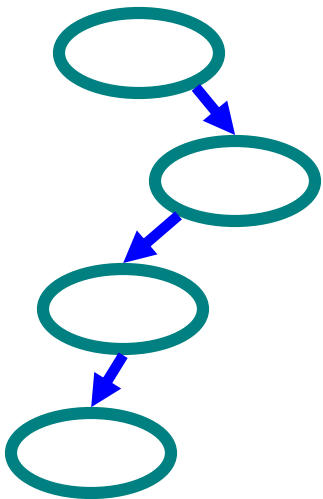
Size and depth



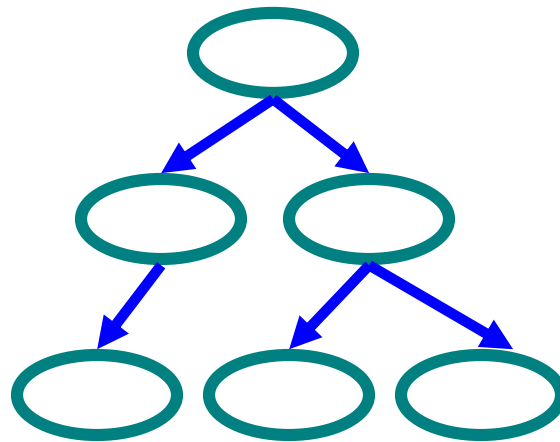
- The **size** of a binary tree is the number of nodes in it
 - This tree has size 12
- The **level/depth** of a node is its distance from the root
 - **a** is at **level** zero
 - **e** is at **level** 2
- The **depth** of a binary tree is the level of its deepest node
 - This tree has depth 4

Types of Binary Trees

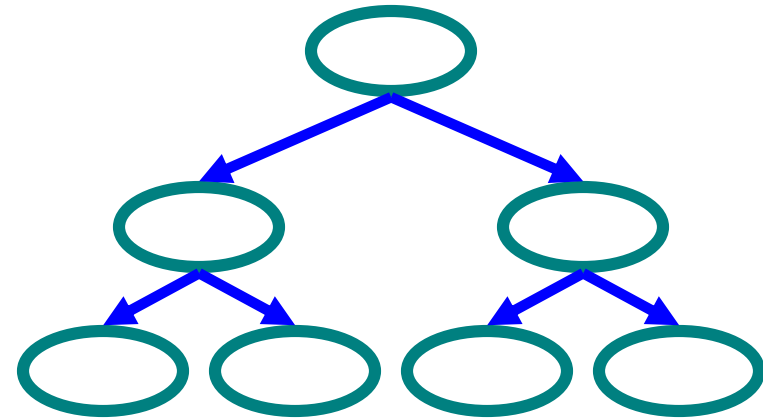
- Degenerate – only one child
- Complete/Full – always two children
- Balanced – “mostly” two children
 - more formal definitions exist, above are intuitive ideas



**Degenerate
binary tree**



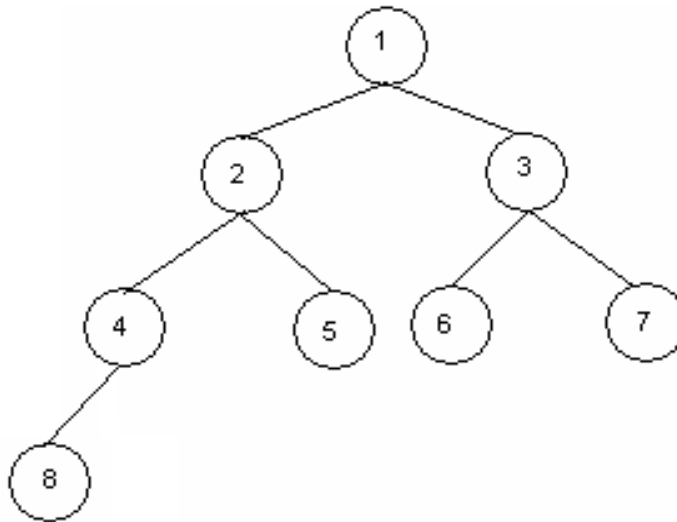
**Balanced
binary tree**



**Complete
binary tree**

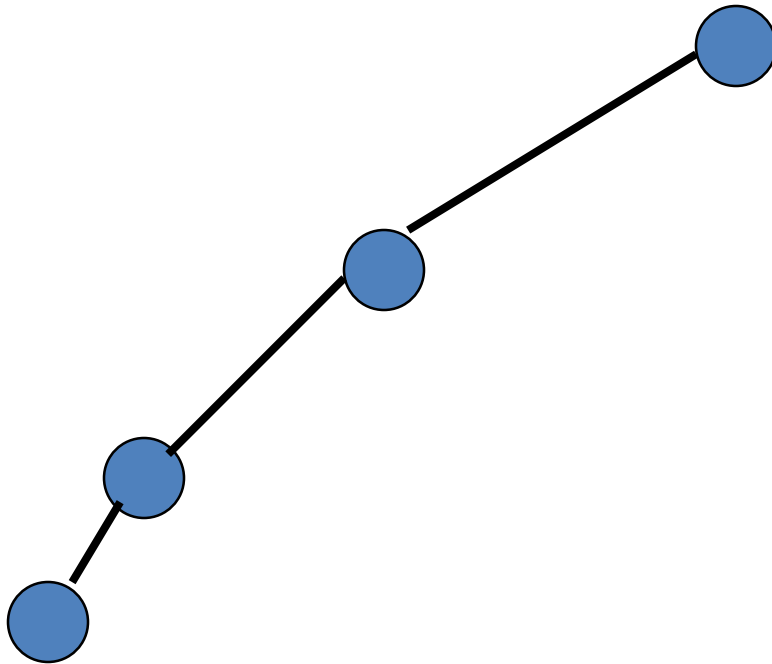
Complete Binary Tree

- All its level, except possibly the last, have the maximum number of possible nodes
- All the nodes at the last level appear as far left as possible.
- The depth D_n of the complete binary tree with n nodes,
$$D_n = \text{Floor}(\log_2 n)$$



Minimum Number Of Nodes

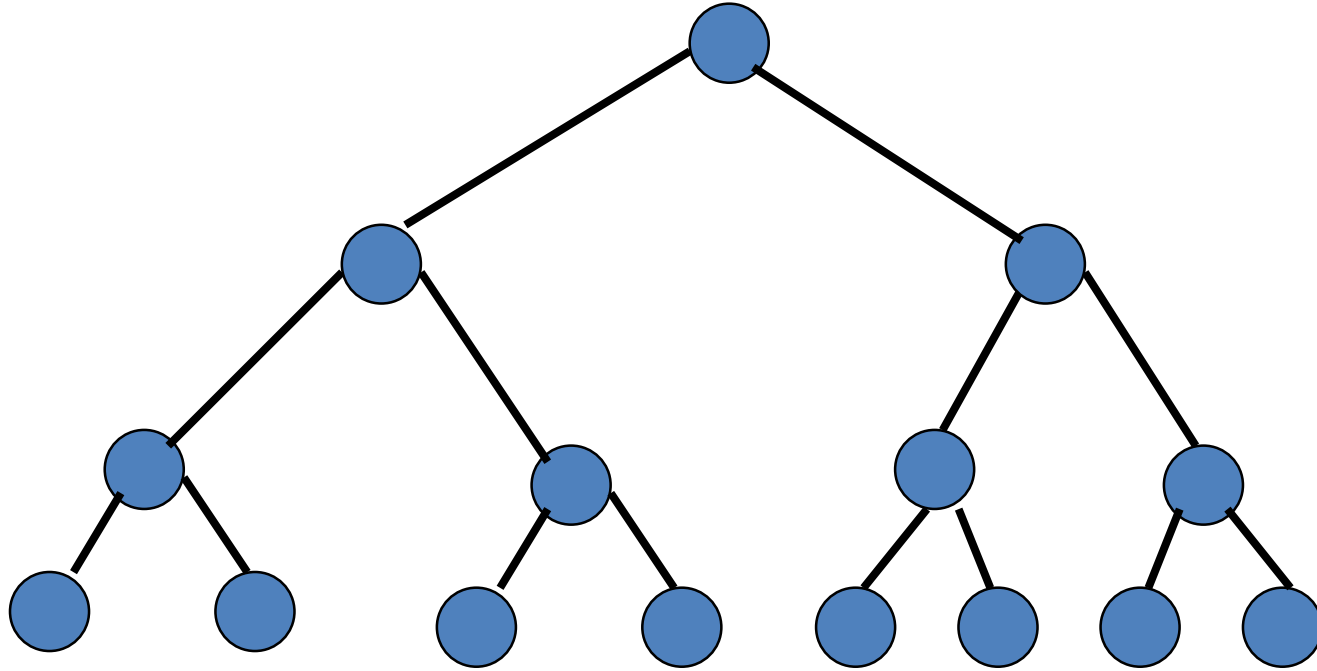
- Minimum number of nodes in a binary tree whose height is h .
- At least one node at each of first h levels.



• minimum number of nodes is h

Maximum Number Of Nodes

All possible nodes at first h levels are present.

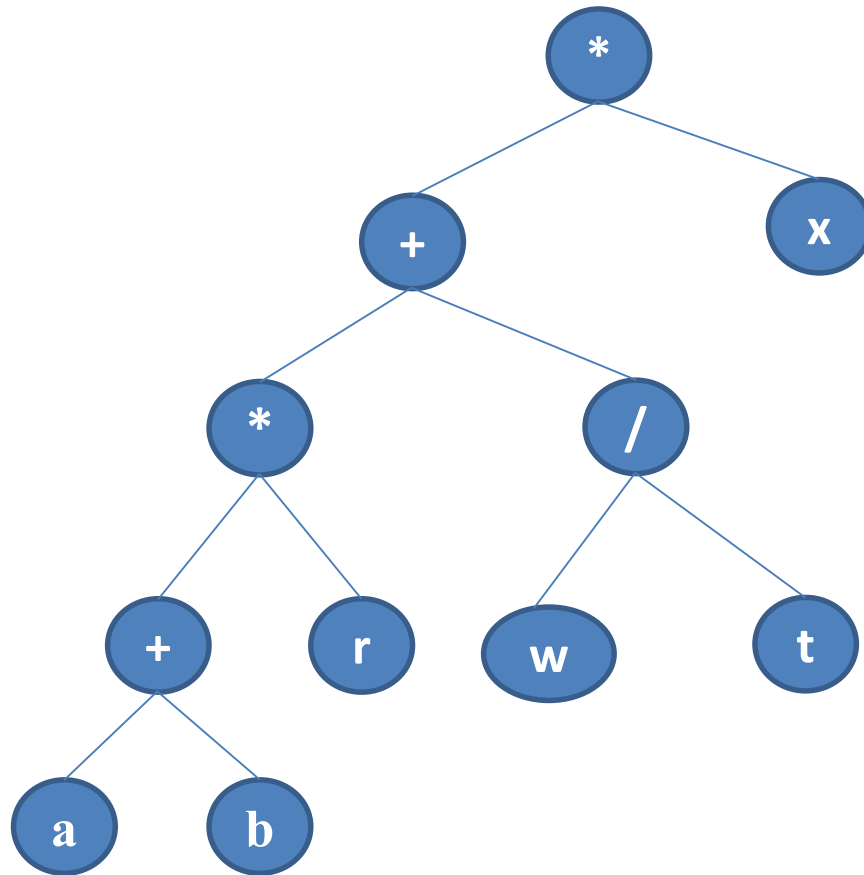


Maximum number of nodes

$$= 1 + 2 + 4 + 8 + \dots + 2^{h-1} = 2^h - 1$$

Representation of Algebraic Expression Using Binary Tree

$$E = ((a + b) * r + w / t) * x$$

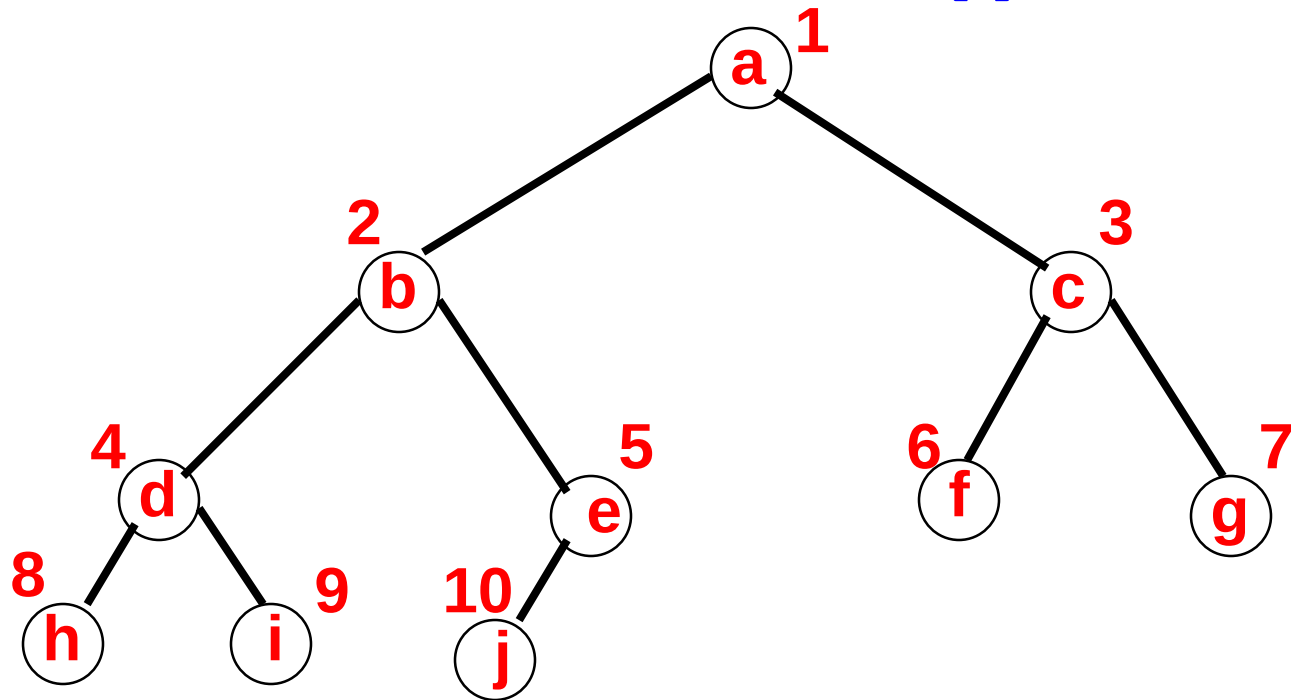


Binary Tree Representation

- Array representation.
- Linked representation.

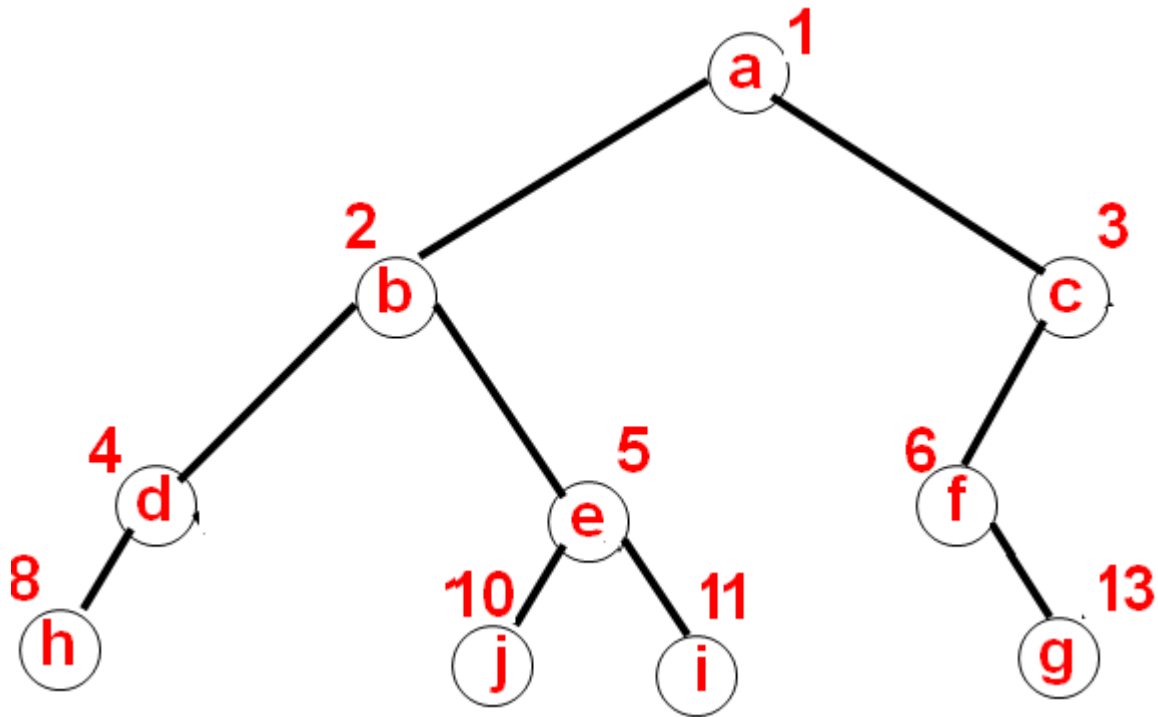
Array Representation

- Number the nodes using the numbering scheme for a full binary tree. The node that is numbered i is stored in `tree[i]`.



tree[]		a	b	c	d	e	f	g	h	i	j
	0					5					10

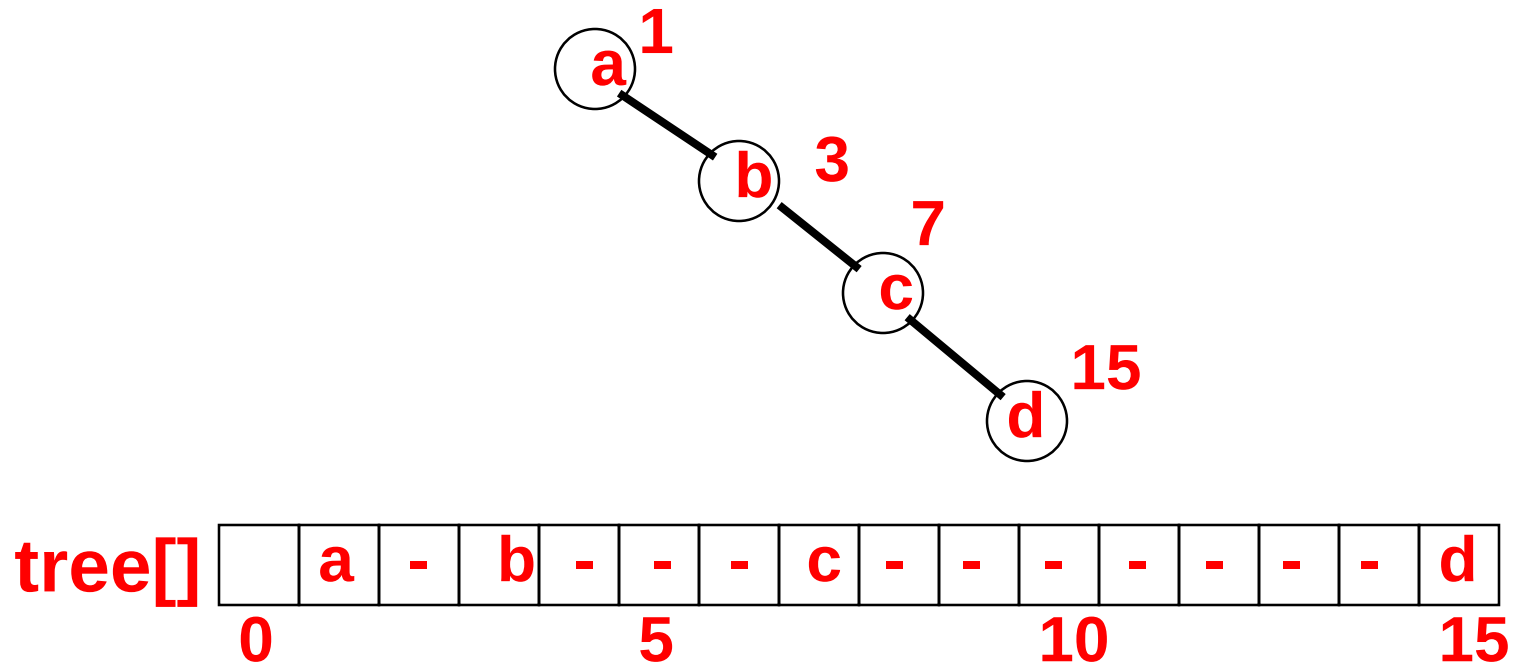
Array Representation Example



tree[]

	a	b	c	d	e	f		h		j	i		g
0	1	2	3	4	5	6	7	8	9	10	11	12	13

Right-Skewed Binary Tree



- An n node binary tree needs an array whose length is between $n+1$ and 2^n .

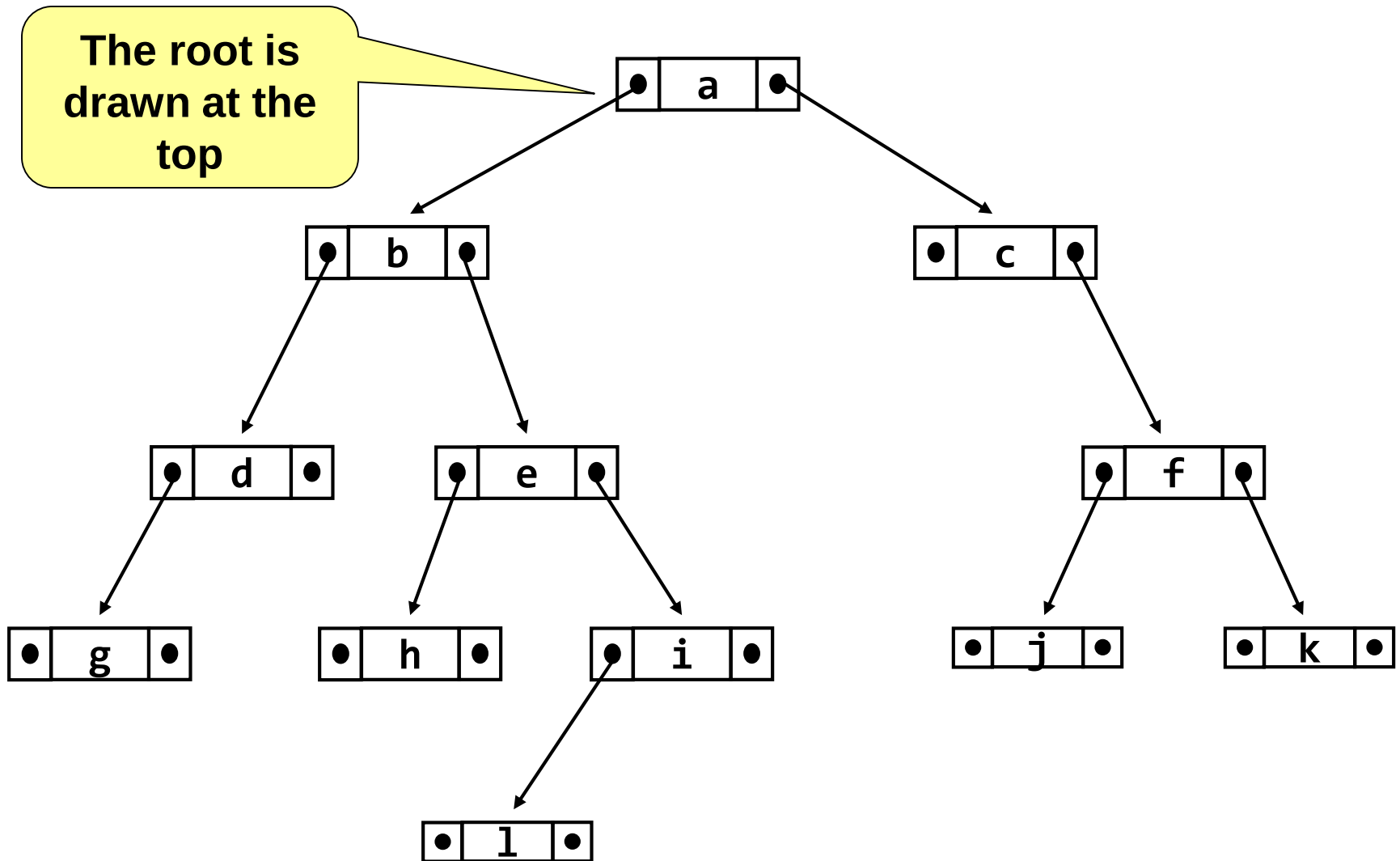
Linked Representation

- Each binary tree node is represented as an object whose data type is `BinaryTreeNode`.
- The space required by an n node binary tree is $n * (\text{space required by one node})$.

The Class BinaryTreeNode

```
class BinaryTreeNode{  
    int data;  
    BinaryTreeNode *leftChild;  
    BinaryTreeNode *rightChild;  
};
```

Picture of a binary tree



Binary Tree Traversal

- Many binary tree operations are done by performing a **traversal** of the binary tree.
- In a traversal, each element of the binary tree is **visited** exactly once.
- During the **visit** of an element, all action (make a clone, display, evaluate the operator, etc.) with respect to this element is taken.

Binary Tree Traversal Methods

- Preorder
- Inorder
- Postorder
- Level order

Tree traversals

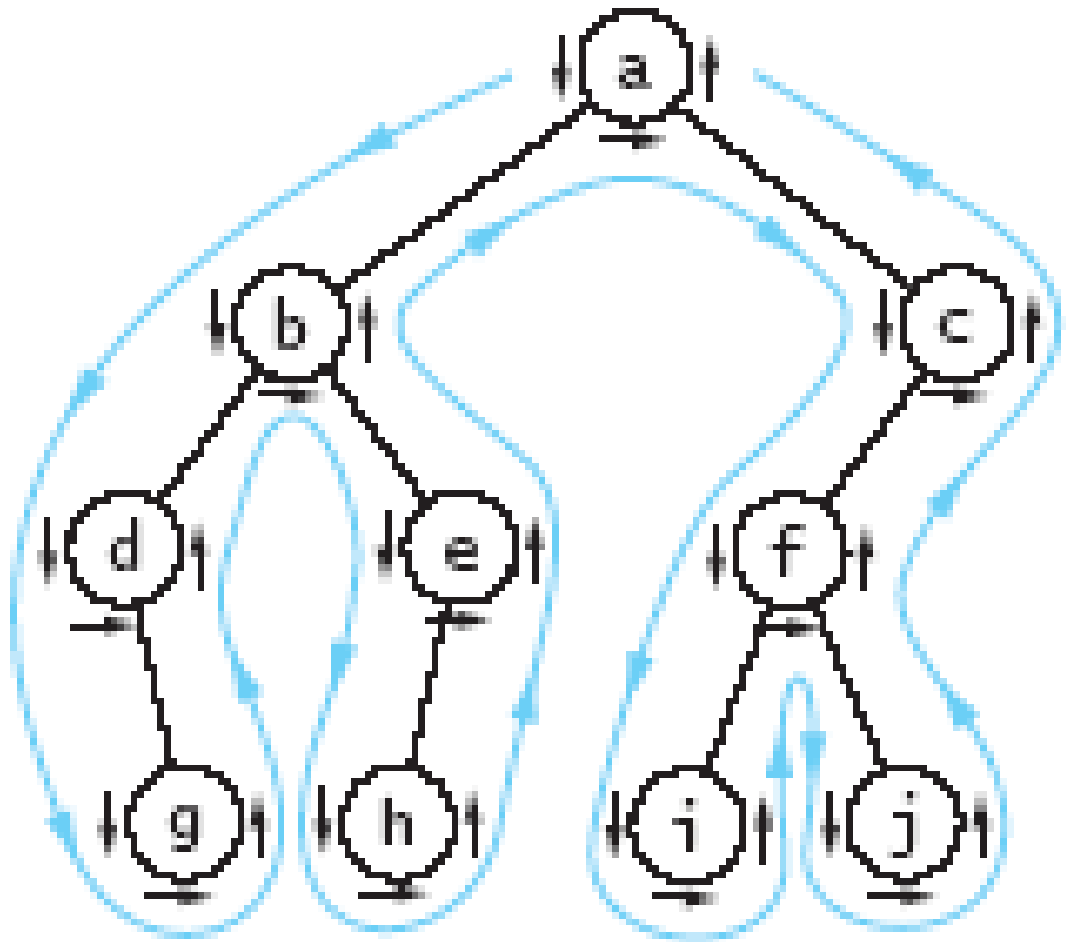
- A binary tree is defined recursively: it consists of a **root**, a **left subtree**, and a **right subtree**
- To **traverse** (or **walk**) the binary tree is to visit each node in the binary tree exactly once
- **Tree traversals are naturally recursive**
- Since a binary tree has three “parts,” there are six possible ways to traverse the binary tree:
 - **root, left, right**
 - **left, root, right**
 - **left, right, root**

Visualizing Tree Traversals (2)

FIGURE 8.9

Traversal of a Binary Tree

- Preorder:
- a, b, d, g, e,
- h, c, f, i, j

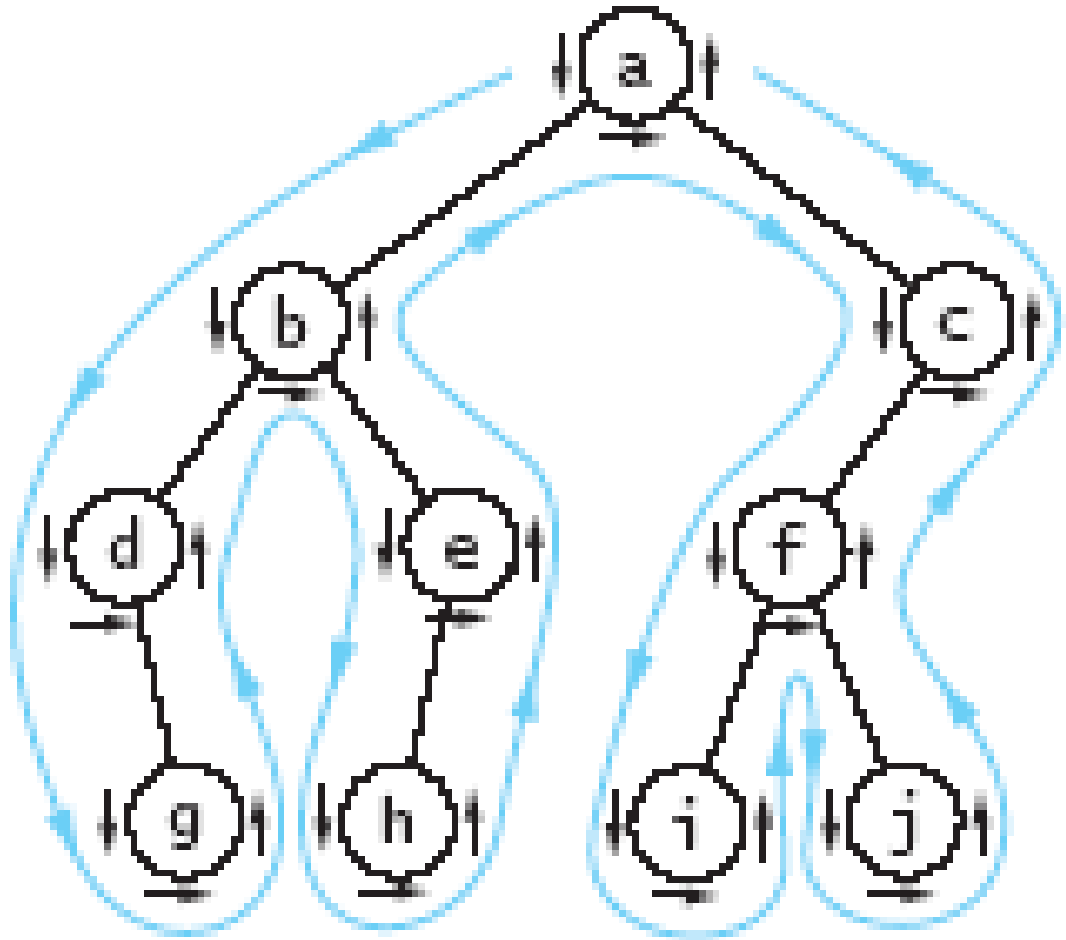


Visualizing Tree Traversals (3)

FIGURE 8.9

Traversal of a Binary Tree

- Inorder:
- d, g, b, h, e,
- a, i, f, j, c

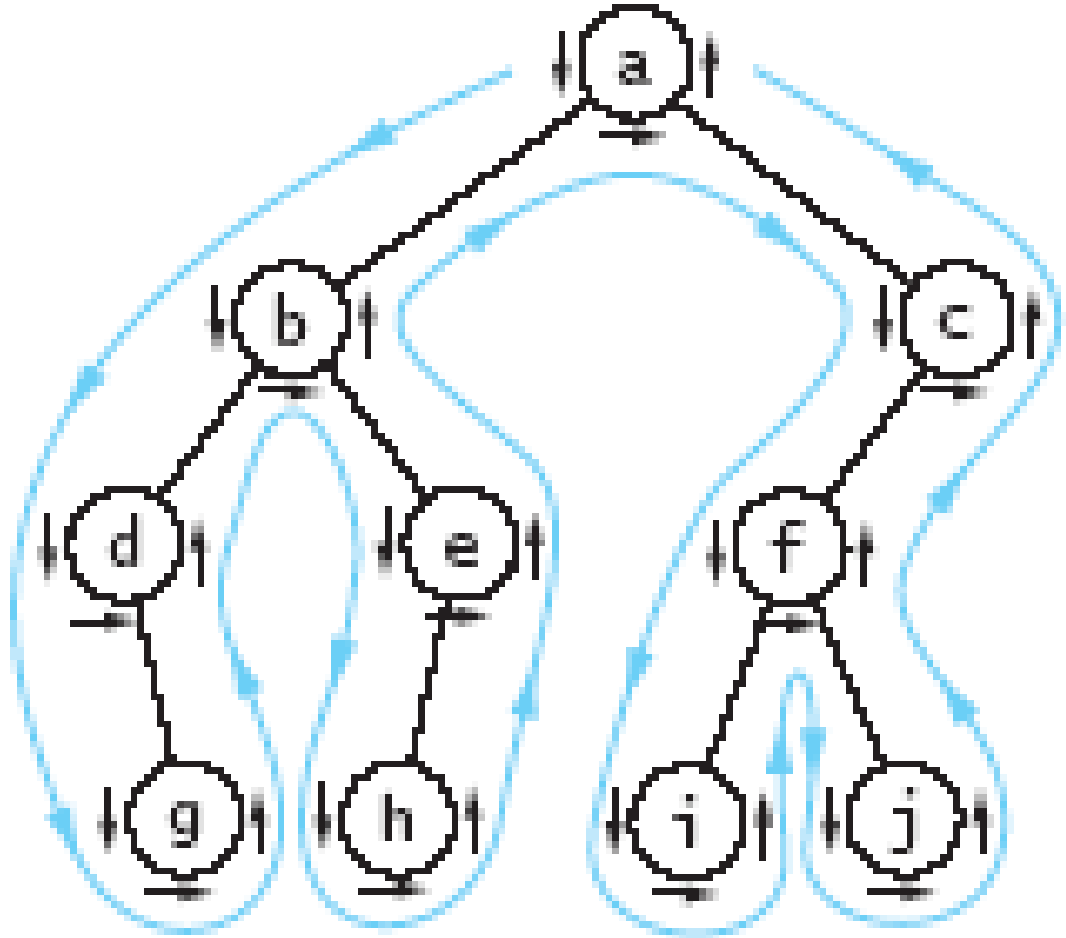


Visualizing Tree Traversals (4)

FIGURE 8.9

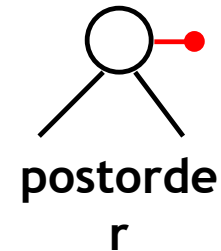
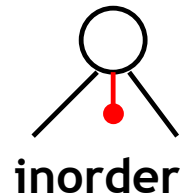
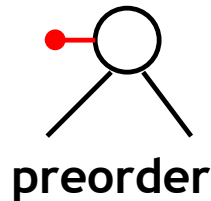
Traversal of a Binary Tree

- Postorder:
- g, d, h, e, b,
- i, j, f, c, a



Tree traversals using “flags”

- The order in which the nodes are visited during a tree traversal can be easily determined by imagining there is a “flag” attached to each node, as follows:



To traverse the tree, collect the flags:

